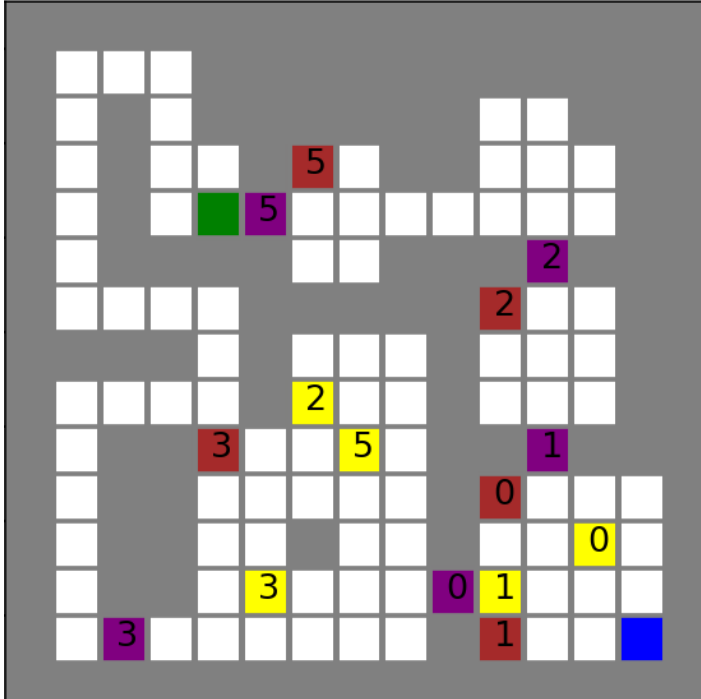


Home assignment 3: RL

Introduction

The point of the exercise is to experiment with solving a problem using RL principles that were taught in class.



The problem is similar to the problem in the previous assignment; the only change is that now the stochastic environment is not known. This assignment will ask you to create a policy offline using RL principles and you will send your RL policy file to be tested. This document will address the changes/additions that are relevant to this assignment.

Problem:

Input: The input to every problem is similar to the previous assignment.

Possible actions: The actions behave similarly to the previous assignment except that now you will not know the environment (the probabilities and rewards).

Success: The agent finds a path to the goal.

Failure: No possible path to the goal can be found in the limited number of steps or dead-end.

Assignment:

For this assignment we've attached code for you to use. Most of what's needed is implemented. What you will have to do is implement the Policy class functions so the checker code can be utilized to solve the problem and some code in the check.py file to generate a policy.

ex3.py file has the following functions for you to implement (you could also have helper functions as you see fit):

1. Choose_next_action function – receives a state of the game (which is a tuple of the game map, tuple of the agent position, the amount of steps taken, if the game is finished, if the game was successful) and returns a single action (e.g. "U")
2. __init__ function – receives a file path for the location to save/load the policy file, and a Boolean value to indicate to either train or test the policy. **It is allowed to add parameters to help you generate policies.**
3. save_policy – receives a file path and saves the relevant information for your policy. **It is up to you how to save your policy.**
4. Load_policy – receives a file path and loads the policy file and unpacks it into the policy object. **It is important to be consistent with your save_policy function, if this function fails or improperly loads your policy it will fail your assignment.**

The check.py there might be some places where you would want to add some code, but it will not be checked for grading (**but it will still be submitted**). The locations where you might want to add code:

1. generate_policy – receives a policy file and generates a trained policy file. **You are allowed to add parameters to this function to help your policy generation/training.**
2. It is suggested to create more problems to test on with the help of gameClass.generate_new_game – receives max steps, initial game map, seed, debug mode --- as input and outputs a game object. The list test_games will hold the tuples of (max_steps, initial_game_map, seed) for policy evaluations.

DO NOT CHANGE THE FUNCTIONS NAMES OR INPUT UNLESS STATED OTHERWISE!

Hidden Information:

Unlike before you will not be given the pressure_plate.py file but an encoded file that has the information on the environment (probabilities/rewards). It is similar to the previous file but it has 2 additional functions to your disposal.

```
def play_game(self, policy, game, visualize=True):
    """Execute given policy, for a given number of steps.
    if Visualize = True, prints all steps along execution.
    Returns the reward"""
    total_reward = 0
    i = 0
    self._done = False
    self.generate_new_game(*game, debug=visualize)
    while i < self._max_steps and not self._done:

        move = policy.choose_next_action(self.get_current_state())
        _, reward, self._done = self.submit_next_action(move)

        total_reward += reward

        i += 1
    if visualize:
        self.show_history()

    return total_reward

1 usage (1 dynamic)
def evaluate_policy(self, policy, games, visualize=True):
    return sum([self.play_game(policy, g, visualize) for g in games]) / (1.0 * len(games))
```

Notes:

- You may change the code in check.py but make sure to not rely on it to create your policy, since it will not be used for testing.
- Your policy may not rely on play_game or evaluate_policy or submit_next_action once loaded (after training), but you may use these function while **training only**.
- It is advised to create multiple different problems for training and different ones for testing.
- There is only one environment model which is unknown to you. It is possible to decode the hidden file yourselves but any use of the hidden information will **DISQUALIFY** your assignment be graded at 0. The finished_reward can be assumed to be greater than max_steps, opening_door_reward can be assumed to be lower than finished_reward (and can be negative), and step punishment can be assumed to be lower than finished_reward/max_steps but always negative.
- Please be consistent with your representations (actions and states, e.g. capitalized letters, underscores, etc.). The testing is automatic and even if the code is compiled correctly you will lose points for inconsistencies.
- You will submit a policy file along with other files. **Do not zip the file**. The submit system indicates that the maximum file size you can upload is 50MB, this is your maximal limit.
- **MAKE SURE YOUR POLICY LOADS CORRECTLY, IF IT DOESN'T WHILE GRADING IT WILL BE GRADED AS 0.**
- Please make sure that your code works on both Linux and Windows, and please ask permission in the assignment forum to use any non-built-in libraries/packages (any library/package that you have to perform pip install to use it).

Grading:

Your policy does not have to be optimal, but it must be on average better than the random policy, sub-par policy (which is unknown to you) and on-par policy (which is unknown to you). The assignment is **not** a competitive assignment (you are not graded compared to other students).

The assignment will be checked automatically, and it is important that the correct names of the files, classes and functions are kept.

The grading program will consist of 20 different “PressurePlate” problems of increasing difficulty, each with a different environment model. Your policy must pass each problem under 5 minutes. If you cannot finish the problems in under 5 minutes the best problems will be taken, and the rest will be considered as failures (standing in place – reward = max_steps * step_punishment) and their reward will be added to your average. To reach a maximum grade, as previous assignments, the run time sum of all problems must be under 5 minutes ($p_1 + p_2 + p_3 + \dots + p_{20} = 5$ minutes).

Your code will need to perform better than the on-par policy on **average** to get 100. Meaning that the average reward of your policy on the 20 different problems with different environment models, will need to be higher than the on-par policy to get 100.

Policies that do not pass the on-par policy will get a lower grade based on the distance to the on-par average. Policies that do not pass the sub-par policy but pass the random policy will get a grade between 60 – 80. Policies that do not pass the random policy will receive 0.

You can assume that maximal matrix size is 20x20 and that at most there will be 10 key blocks.

Self-check: The attached file `check.py` has two example problems. It will not check the correctness of code or is enough for sufficient testing. It is encouraged to create more example problems to check the robustness of your code.

Attached files:

The files include:

1. `Ex3.py` – The only file to be submitted and the only one relevant for grading.
2. `check.py` – The file to use for self-checking, any changes here should only be to adding more problems for you to check or any additional information for policy generation (the submission won't check your added problems).
3. `Pressure_plate.encoded` – The file that holds the class of the game assignment, and is hidden from you.

Submission:

- Deadline – 3.7.25
- Solo submissions.
- Please put your ID in the variable `id` in `ex3.py` and along with submitting `ex3.py` please submit a `details.txt` that includes your name (in English only) and your ID (One line after the other). In addition please submit your policy file and your `check.py` file.
- Any questions please use the Lamda forum for the assignment.
- The submission will use the [submit](#) system. Upload `ex3.py`, `details.txt`, `check.py` and your policy file. Do not upload any other files or in any other format (like zip or rar).